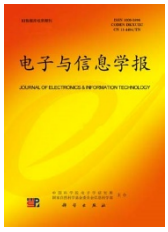


电子与信息学报
Journal of Electronics & Information Technology
ISSN 1009-5896,CN 11-4494/TN

《电子与信息学报》网络首发论文

题目：　面向国产交换芯片的多模态网络后端编译器设计
作者：　涂化清，王元鸿，徐琪，朱俊，邹涛，隆克平
收稿日期：　2025-03-05
网络首发日期：　2025-12-01
引用格式：　涂化清，王元鸿，徐琪，朱俊，邹涛，隆克平．面向国产交换芯片的多模态网络后端编译器设计[J/OL]．电子与信息学报．
https://link.cnki.net/urlid/11.4494.TN.20251128.1117.008

网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

网络首发时间：2025-12-01 07:04:05　网络首发地址：https://link.cnki.net/urlid/11.4494.TN.20251128.1117.008

第47卷 第12期　电 子 与 信 息 学 报　Vol. 47No. 12
2025年12月　Journal of Electronics & Information Technology　Dec. 2025

面向国产交换芯片的多模态网络后端编译器设计

涂化清[①]　王元鸿[②]　徐　琪[①]　朱　俊[①]　邹　涛[*①]　隆克平[①]

[①](之江实验室　杭州　310000)

[②](中国科学院计算技术研究所　北京　100000)

摘　要：当前，后端编译器相关工作主要针对软件可编程交换机(BMv2)、现场可编程门阵列(FPGA)、Intel Tofino系列芯片等可编程设备进行设计和优化，不适用于国产盛科TsingMa.MX交换芯片上多模态网络程序的编译。为此，该文提出面向TsingMa.MX交换芯片的多模态网络后端编译器p4c-TsingMa，实现了高级网络编程语言到TsingMa.MX交换芯片的编译，使TsingMa.MX交换芯片同时支持多种网络模态报文的解析与转发。p4c-TsingMa首先使用先序遍历方法从中间表示中提取出协议类型、协议字段、动作等关键信息，然后根据所提取的信息进行指令转译，最终生成TsingMa.MX芯片控制命令。同时，p4c-TsingMa采用用户自定义字段(UDF)合并方法，将不同网络模态的匹配指令合并在1个查找表中，从而1次提取多个模态的匹配项，提高芯片资源利用率。实验结果表明，p4c-TsingMa可实现对多种网络模态程序的正确编译，相较于未启用 UDF 表项合并算法、单端口独立配置各模态UDF规则的场景，其可将寄存器资源利用率提升37.5%～75%。

关键词：多模态网络；可编程数据平面；国产交换芯片；后端编译器

中图分类号：TN393　**文献标识码**：A　**文章编号**：1009-5896(2025)12-0001-11

DOI: 10.11999/JEIT250132　**CSTR**: 32379.14.JEIT250132

1　引言

信息网络是国家核心基础设施，是世界各国激烈竞争的战略制高点。随着网络规模不断扩大和新兴业务不断涌现，传统网络单一承载、功能固定僵化等问题日益突出。为此，信息工程大学邬江兴院士团队提出多模态网络(PolymorphIc Network, PINet)[1,2]发展范式，采用“技术体系与支撑环境分离[3]”思想，支持各种网络技术体系(网络模态)在同一网络环境(支撑环境)中动态部署和共存，由多元化的网络模态提供差异化、定制化的服务能力，从而满足垂直行业的多样化用网需求，已成为我国科技自主创新和发展的重要内容[4]。

为了在同一物理环境中支撑多种网络模态的共生共存和独立演进，多模态网络在具体部署环境中使用可编程交换机与协议无关的数据包处理器编程(Programming Protocol-independent Packet Processors, P4)[5]。可编程交换机采用的可编程芯片能够通过编程实现多种网络协议数据包的处理。目前国内外厂商都纷纷推出了可编程交换芯片。国外方面，美国Barefoot公司于2016年发布了世界上第1款完全具备可编程能力的交换芯片Tofino。Intel公司收购该公司后继续发布了Tofino2以及Tofino3[6]。国内方面，盛科通信作为国内领先的以太网交换芯片设计企业推出了TsingMa.MX [7]可编程芯片，支持2.4T I/O带宽，提供从1 000M～400G的全速率端口能力。采用国产可编程交换芯片搭建多模态网络基础设施，研发具有独立自主知识产权的多模态网络编程系统，有助于我国高水平自立自强、可持续发展特色的通信网络发展。P4程序在可编程交换芯片上运行需要先后经过前端编译器和后端编译器的编译。前端编译器将P4程序转换为中间表示形式，然后后端编译器将中间表示翻译成可编程交换机软件平台所需的指令和配置。尽管P4程序的前端编译器具备通用性，能够适用于多种环境，但与之相对的是，各种可编程设备的后端编译器则不具备这种通用性，它们通常是针对特定设备或平台进行优化和定制的。

虽然当前有很多后端编译器优化相关的工作[8–15]，但是它们主要针对的是软件可编程交换机(Behavior Model version 2, BMv2)、现场可编程门阵列(Field Programmable Gate Array, FPGA)、Intel Tofino系列芯片等可编程设备设计后端编译器。不同编译方法对应的目标平台在底层架构设计上遵循完全不同的技术路线，因此当前后端编译器

收稿日期：2025-03-05；改回日期：2025-10-16

*通信作者：　邹涛　zout@zhejianglab.org

基金项目：国家重点研发计划(2022YFB2901503)，国家自然科学基金(U22A2005)，浙江省重点研发计划(2024SSYS0001)，中国博士后科学基金(2024M75986)。

Foundation Items: The National Key Research and Development Project of China (2022YFB2901503), The National Natural Science Foundation of China (U22A2005), The Key R&D Program of Zhejiang (2024SSYS0001), China Postdoctoral Science Foundation (024M75986).

相关工作不适用于国产TsingMa.MX交换芯片。TsingMa.MX芯片采用非协议无关交换架构(Protocol Independent Switch Architecture, PISA)架构，以灵活解析/灵活编辑(FlexParser/FlexEdit)模块为核心实现可编程能力，硬件层面存在单端口最多2个查找表、单查找表仅1个128 bit 寄存器的严格资源约束，且无原生P4语言支持，需通过自定义映射规则实现编译适配；而BMv2作为软件交换机，本质是基于通用中央处理器(Central Processing Unit, CPU)的逻辑模拟环境，无真实硬件资源限制；FPGA虽为硬件加速平台，但侧重并行计算单元的灵活配置，与交换芯片的报文解析/转发硬件结构差异显著；Tofino系列芯片则是典型的PISA架构，具备原生P4支持的匹配-动作流水线，编译适配可直接复用架构特性。这种从硬件指令集到资源分配逻辑的底层差异，使各编译器的技术实现路径完全不同。

本文针对盛科TsingMa.MX交换芯片需设计专有多模态网络后端编译器p4c-TsingMa。针对设计过程中存在的处理复杂有向无环图(Directed Acyclic Graph, DAG)中间表示解析、适配芯片特性映射、突破寄存器资源限制3大挑战，p4c-TsingMa首先对中间表示采用先序遍历，根据中间表示(Intermediate Representation, IR)中的节点类型，将DAG中的结点信息，合并为特定数据结构保存。然后，根据特定数据结构和TsingMa.MX芯片的FlexParser和FlexEdit特性，计算出FlexParser和FlexEdit配置的资源映射。最后，在进行FlexParser资源映射时，根据芯片的资源限制，提出了用户自定义字段(User Defined Field, UDF)表项合并算法，来解决寄存器资源有限的问题。本文将针对盛科TsingMa芯片设计的后端编译器p4c-TsingMa与适配BMv2, FPGA, Tofino目标平台的编译器，从IR解析策略、硬件映射逻辑及核心优化方向这3个层面展开对比，对比总结如表1所示。

2 TsingMa.MX交换芯片的后端编译器设计

2.1 后端编译器架构

后端编译器p4c-TsingMa的整体架构如图1所示。P4源程序经过开源的p4c[19]编译器前端和中间端优化之后，将中间表示文件传递给后端编译模块，后端编译模块首先对中间表示进行解析(3.2节)。解析完成之后，根据TsingMa.MX交换芯片的实际报文处理流水线以及芯片的UDF,FlexParser和FlexEdit能力，进行程序转译(3.3节、3.4节、3.5节)。最后将程序转译之后的结果生成针对TsingMa.MX芯片的指令序列(3.6节)。当硬件平台拿到编译之后的指令序列后，使用交换芯片的抽象接口，对芯片进行配置，从而使TsingMa.MX交换芯片满足多模态网络环境的数据报文处理要求。

2.2 中间表示解析

中间表示由P4代码经p4c编译器前、中端优化生成，作为后端编译器的输入，使其无需关注词法分析、语法分析和目标无关的语义检查。它以DAG结构存储，涵盖P4程序中的所有变量定义、函数及逻辑关系。p4c前端编译器提供访问者(Visitor)模式，支持先序遍历DAG节点。每个节点对应P4程序中的变量、函数等内容，属性包括节点标识符(Node_ID)、节点类型(Node_Type)、名称(name)、节点体(body)等。不同P4语句对应不同节点类型，解析时主要关注头部类型(Type_Header)、结构体类型(Type_Struct)、解析器状态(ParserState)、P4动作(P4Action)和P4表(P4Table)，分别对应V1Model模型中的头部(header)、结构体(struct)、解析器(parser)、动作(action)和表(table)。本文以头部节点类型为例，说明P4源程序与中间表示的对应关系及节点解析方法，其他类型节点解析类似，不再赘述。

图2(a)给出了P4语言定义的以太网头部，其中包括48位的目的地址字段(dstAddr)、48位的源地址字段(srcAddr)以及16位的以太网头部类型字段

表 1 不同编译方法对比

目标平台	编译方法	IR解析策略	硬件映射逻辑	核心优化方向
BMv2	P4LLVM[16]	利用底层虚拟机框架解析P4代码，提取可通过底层虚拟机优化通道处理的网络程序逻辑	生成JSON格式输出，借助底层虚拟机的通用优化能力提升网络程序效率	为软件交换机bmv2提供专用后端编译支持
FPGA	P4FPGA[17]	解析P4代码并提取适合FPGA硬件实现的并行、高性能逻辑	生成Verilog代码，简化FPGA开发，融合P4的灵活性与FPGA的高性能	借助底层虚拟机优化通道提升网络程序执行效率
Tofino	Chipmunk[18]	解析P4代码并进行多层级编译处理，抽象掉Tofino硬件特定细节	生成Tofino专用文件，提供高层级编程抽象，适配Tofino的匹配 - 动作流水线架构	简化FPGA开发，实现P4与FPGA的高效融合
TsingMa	p4c-TsingMa (本文方法)	通过先序遍历解析DAG结构的中间表示，精准提取协议类型等关键信息	自定义P4 语义-国产芯片 FlexParser / FlexEdit硬件参数”映射规则，生成适配的芯片指令，解决非原生P4支持的硬件适配问题	提供高层级编程抽象，屏蔽Tofino硬件细节

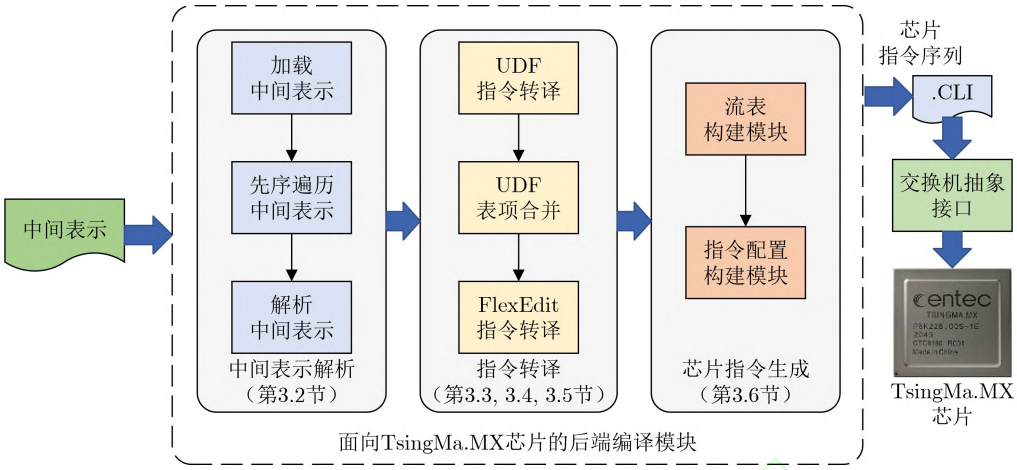


图 1 后端编译器p4c-TsingMa.MX的整体架构

```
1  header ethernet_t {
2      bit<48> dstAddr;
3      bit<48> srcAddr;
4      bit<16> ether_type;
5  }
```

(a) P4语言定义的以太网头部

```
1  {
2      "Node_ID" : 1374,
3      "Node_Type" : "Type_Header",
4      "name" : "ethernet_t",
5      "declid" : 64,
6      "annotations" : { ...
7      },
8      "typeParameters" : { ...
9      },
10     "fields" : { ...
11     },
12     "Source_Info" : { ...
13     }
14 }
```

(b) 以太网头部的中间表示

```
1  "fields" : {
2      "Node_ID" : 1375,
3      "Node_Type" : "IndexedVector<StructField>",
4      "vec" : [
5        {
6          "Node_ID" : 1363,
7          "Node_Type" : "StructField",
8          "name" : "dstAddr",
9          "declid" : 197,
10         "annotations" : {
11           "Node_ID" : 5
12         },
13         "type" : {
14           "Node_ID" : 1362,
15           "Node_Type" : "Type_Bits",
16           "size" : 48,
17           "isSigned" : false,
18         ... },
19         ... },
20 ... ],}
```

(c) 域子节点

图 2 P4源程序及IR比较

(ether_type)。经过前端编译之后的中间表示如图2(b)所示，其中节点标识符(Node_ID)表示该节点在中间表示中的唯一标识，节点类型(Node_Type)表示该节点的类型为头部类型(Typer_Header)，节点名字(name)表示该节点对应的源代码中的变量名，源代码信息(Source_Info)表示该变量在源代码中的位置，域(fields)是一个子节点，里面包括了以太网头部内部定义的其他字段。将域子节点展开，如图2(c)所示，可以看到它是一个向量类型(IndexedVector)的存储结构，保存了头部类型节点中3个变量dstAddr,srcAddr和ether_type的具体信息。将该向量属性展开，可知节点名字为dstAddr与P4代码中ethernet_t的第1个变量对应。类型属性描述了变量dstAddr的类型是Type_Bits，长度是48。

算法1实现了P4协议头的结构化偏移量计算。该算法采用协议头定义顺序进行线性遍历，以贴合P4语言中header类型字段的声明逻辑(即字段在代码中的定义顺序与报文头部的实际存储顺序一致)。例如，在以太网头部ethernet_t中，dstAddr、srcAddr、ether_type的声明顺序与报文帧中的字段排列顺序严格对应，因此按此顺序遍历可直接映射物理报文结构，避免额外的顺序调整开销。同时，该算法采用动态偏移量累加机制，逐层构建包含字段偏移、位宽和类型的元数据集合。动态偏移量累加的具体实现逻辑是针对P4语言支持的固定位宽(Type_Bits)与枚举类型(Type_Enum)等字段类型，算法在遍历中增加类型适配判断：若字段类型为Type_Enum，则自动提取其底层映射的位宽(如8位、16位)后纳入偏移量累加；若为Type_Bits，则直接使用声明的位宽值。该设计确保了对P4标准字段类型的全覆盖，整体时间复杂度稳定为 $O(n)$(n为协议头字段数量)。

算法 1 协议头字段偏移量计算算法

输入：hdr-P4协议头类型定义(Type_Header对象)

输出：更新全局映射structOffsets，存储协议头字段的元数据

(1) 初始化 structInfo← ∅

(2) headerName←*hdr*的名字

(3) 创建空的headerOffset字典

(4) offset← 0

(5) **for** each 字段 ele **in** hdr 的字段列表 **do**：

(6) structInfo.eleName←ele.name

(7) **if** ele的类型是 Type_Bits **then**

(8) structInfo.eleType, length← "Type_Bits",ele的位宽

(9) offset←offset+ele的位宽

(10) **end if**

(11) 将(ele的名称，structInfo) 插入 headerOffset 字典

(12) **end for**

(13) structInfo.eleName, length, eleOffset← "totalOffset",offset,offset

(14) 将 ("totalOffset",structInfo) 插入 headerOffset字典

(15) 将 (headerName, headerOffset) 插入全局 structOffsets字典

此外，从结构体类型节点中，提取出定义的所有报文头部协议类型(协议类型信息)，如IPv4，IPv6等。从解析器节点中，提取出3层协议的字段相较于2层协议的偏移(协议字段信息)。从表节点中，提取出匹配操作中需要用到表名和字段名(表、关键字信息)。从动作节点中，提取出对报文的操作动作(动作信息)。

2.3 UDF指令转译

在对中间表示进行解析后，需将解析结果转译成UDF指令(即配置UDF规则的指令)。通常情况下交换芯片识别报文的逻辑是固定的，只支持匹配固定报文格式中的固定位置的协议字段，并且不支持数据包有效荷载部分的内容匹配。在一些应用情况下需要处理芯片设计时没有支持的协议报文。为了解决这样的问题，盛科TsingMa.MX芯片设计了UDF的方法，可自定义报文字段，按照自定义的偏移量提取包头特定字段，结合ACL等功能可达到处理特定报文的目的。

为利用TsingMa.MX交换芯片的UDF功能，需将中间表示解析结果转译成UDF指令，以在入口处理引擎阶段自定义具体的UDF规则，包括具体的要识别的协议头部字段，以及匹配之后对数据包的操作。编译之后的每条UDF规则代表一种模态的报文处理方式，对应P4源程序中的一个表(Table)，每个表包括关键字(Key)和动作(Action)2个部分，关键字包括具体的要匹配的字段信息，例如偏移量、长度、起始偏移位置、匹配值等。动作包括对数据包的具体操作，例如转发还是丢弃数据包。以用户自定义的IPv4协议的UDF规则为例，关键字主要包括两个字段的信息，即目的IP地址信息(dst_addr)和2层协议类型信息(etherType)。当芯片匹配字段成功之后，就会对该数据报文进行相应动作中的处理。

UDF指令转译过程如图3所示。首先从表、关键字信息中提取出表名和关键字名，从协议类型信息中查找关键字对应的协议类型信息，然后根据协议字段信息，计算出字段长度、偏移量和字段值。最后，按照表名、关键字名以及关键字的详细内容

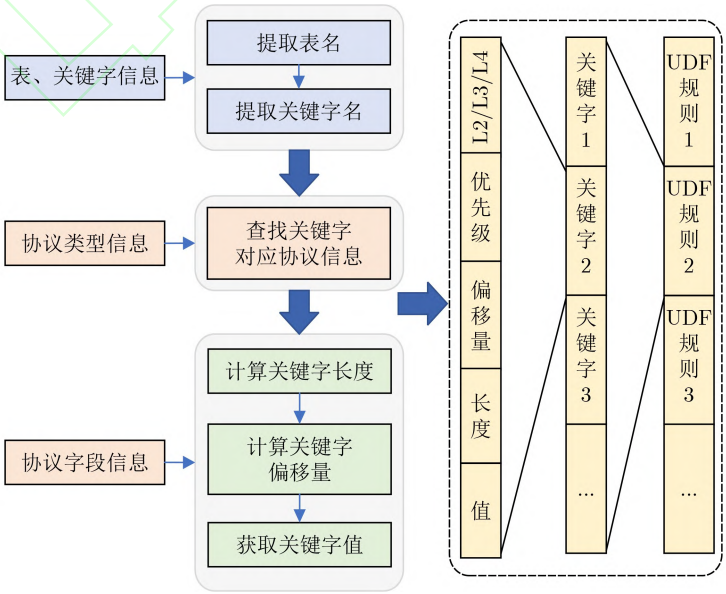


图 3 UDF指令转译过程

的结构，组合成一条完整的UDF规则，其结构如图3所示。

2.4 UDF表项合并

TsingMa.MX芯片使用查找表来存储UDF规则，然而芯片上的查找表资源稀缺，能存储的UDF规则数量有限。根据芯片资源限制，1个端口仅能同时下发2条并行的UDF规则，每条规则关键字的提取长度为128 bit。正常情况下，1条UDF规则代表1种模态的数据报文处理方式。虽然可以通过下发多条串行的UDF规则来实现各个模态报文的处理，但是后续下发UDF规则时，就需要把之前配置的UDF规则进行清空。然而，在实际的多模态网络环境中，这种串行方式并不能满足多模态网络环境同时处理多种模态报文的要求。如果采用并行的方式下发UDF规则，那么按照1条规则处理1种模态的报文，单次又仅能下发2种模态的UDF规则，并不能同时处理多于2种模态的数据报文。

因此，为了使TsingMa.MX芯片具备同时处理多种模态报文的能力，本文提出将UDF规则进行合并编译的方法，通过软件的方式来弥补硬件资源的不足。UDF规则合并编译的思想是，将不同网络模态的匹配指令合并在1个查找表中，从而1次提取多个模态的匹配项。多种模态的多条UDF规则通过编译合并为1条UDF规则，从而使芯片可以同时处理多种模态的数据报文。图4以IPv4、核心级移动标识(Core Mobility First, CoreMF)及工控标识(PowerLink, PwL)3种网络模态为例，展示了UDF 规则合并前后的关键字具体位分布情况。从中可见，3种模态的 UDF 规则均需提取报文头部中同一位置的 etherType 字段作为基础协议标识，并结合1个或多个不同偏移位置的自定义字段(如IPv4的dst_addr、CoreMF的dst_rid、PwL的 dstAddr)完成数据报文识别。由于不同模态的自定义字段分布于报文的不同偏移区域，且绝大多数模态的关键字总长度未超过TsingMa.MX芯片128 bit的寄存器位宽限制，因此可将多模态UDF规则的关键字整合合并。这种表项合并方式不仅突破了芯片单端口仅能并行配置两种模态规则的硬件限制，还能通过减少字段冗余存储(如复用ether-Type字段)显著提升寄存器资源利用率。

算法2给出多个模态的UDF规则合并过程，它将不同模态的UDF规则集合、合并之后规定的UDF规则数量和硬件资源支持的每条UDF规则最大长度作为输入，算法输出为合并之后的UDF规则集合。该算法采用启发式算法结合降序排序分组策略，平衡每个指令组的长度，将不同模态的多条匹配指令合并在1个寄存器中，从而1次配置多个模态的匹配项。采用降序排序的原因是可以优先处理关键字长度更长的UDF规则，避免长规则因后期分配时剩余寄存器资源不足而单独占用1个分组，从而减少分组浪费、提升整体资源利用率。同时，针对含相同关键字字段(如etherType)的规则，算法在选择分组时会优先匹配已含该字段的分组，通过字段复用减少重复提取导致的位宽消耗。例如，AccMF与CoreMF均含etherType，合并后仅需存储1次etherType的16 bit，节省16 bit资源。算法的具体步骤如下：首先将UDF规则根据关键字长度进行降序排序，然后设置mergeNum个模态分组，将每组初始化为∅，再遍历排序后的UDF规则，依次将UDF规则加入消耗存储资源最少的模态分组，然后遍历模态分组，将每个分组中的UDF规则构建为新的UDF规则，最后将合并之后的UDF规则返回，算法结束。

2.5 FlexEdit指令转译

TsingMa.MX交换芯片的FlexEdit能力，主要体现在对数据报文进行灵活的编辑和操作，可以从报文的2层/3层/4层头开始，在指定偏移位置插入/删除/修改指定长度的数据报文段，结合Flex-Parser能力，可灵活地对数据报文进行编辑操作。为了在多模态网络环境中利用TsingMa芯片对报文

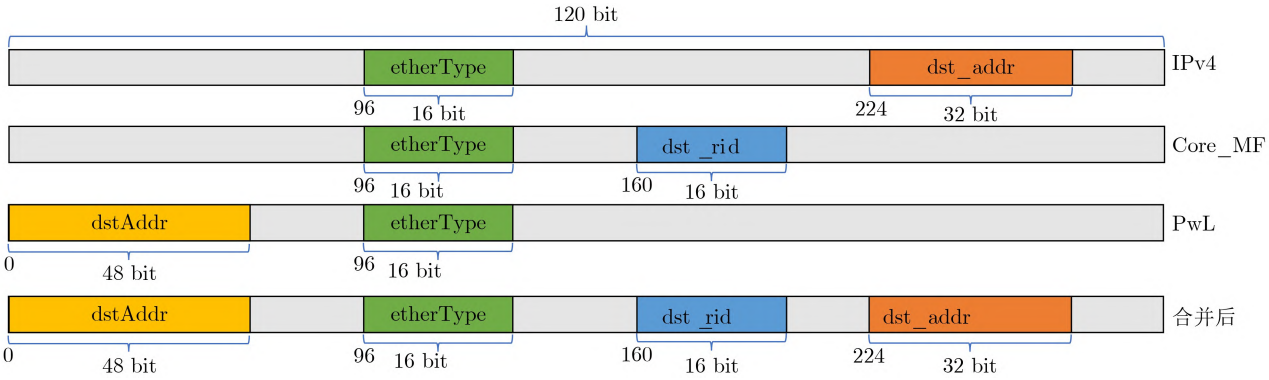


图 4 关键字具体位分布图

算法 2 UDF表项合并算法

(1) 对ruleSet中的UDF规则根据关键字长度进行降序排序
(2) 设置mergeNum个模态分组，每组初始化为∅
(3) **for** each UDF规则in ruleSet：
(4) 找到消耗存储资源最少的模态分组，记为G_min
(5) G_min ← G_min∪{UDF规则}
(6) **end for**
(7) **for each** 模态分组：
(8) 构建合并的UDF规则mergeRule
(9) mergeRuleSet ← mergeRuleSet ∪ mergeRule
(10) **end for**
(11) **return** mergeRuleSet

灵活编辑的能力，对多模态的数据报文进行编辑操作。P4程序使用内置函数添加新的协议字段，通过动作函数中的参数传递添加的字段值，也可以使用内置函数删除协议字段。TsingMa.MX芯片的FlexEdit指令反映了上述P4程序数据包包头编辑逻辑。

FlexEdit操作的指令转译过程如图5所示。首先从中间表示解析得到的动作信息中提取出动作类型和操作字段名，然后根据操作字段名查找协议类型信息，查找出字段对应的协议信息、协议偏移量和协议长度，并查找协议字段信息，计算出编辑字段的长度、偏移量和字段值。最后计算编辑字段前后位置的偏移量、长度和值，将编辑字段及前后位置的偏移量、长度和字段内容组合为编辑操作的协议头部值，从而组合成一个编辑操作的动作指令，其结构如图5右侧所示。

2.6 芯片指令生成

在指令转译阶段，只是将P4程序描述的数据报文匹配和编辑规则进行了转译，转译的结果只包含偏移位置及一些特性的字段，不能对实际的数据报文进行操作。因此，在后端编译最后一个阶段，芯片指令序列生成模块将指令转译的结果与流表文件相结合，构建交换机平台命令行接口(Command-Line Interface, CLI)，从而使交换机能够处理实际的报文。

指令配置构建模块在云开放网络软件(Software for Open Networking In the Cloud, SONIC)系统的上层通过代理服务的方式实现。该代理服务获取模态配置流表文件以及交换平台系统、芯片、端口情况等设备信息，并将编译生成的指令序列与上层应用程序下发的流表转换成相应的交换机平台命令CLI。这样，上层应用程序就可以屏蔽不同交换平台的硬件以及命令行界面的差异，适应不同的底层ASIC芯片。以IPv4模态UDF规则的指令配置构建为例，根据UDF转译的结果与表2描述的流表，构建如表2所示的3个CLI命令，并将3个CLI命令配置下发到芯片上，芯片就可以根据这3条命令就能完成对IPv4模态报文的匹配和转发操作。

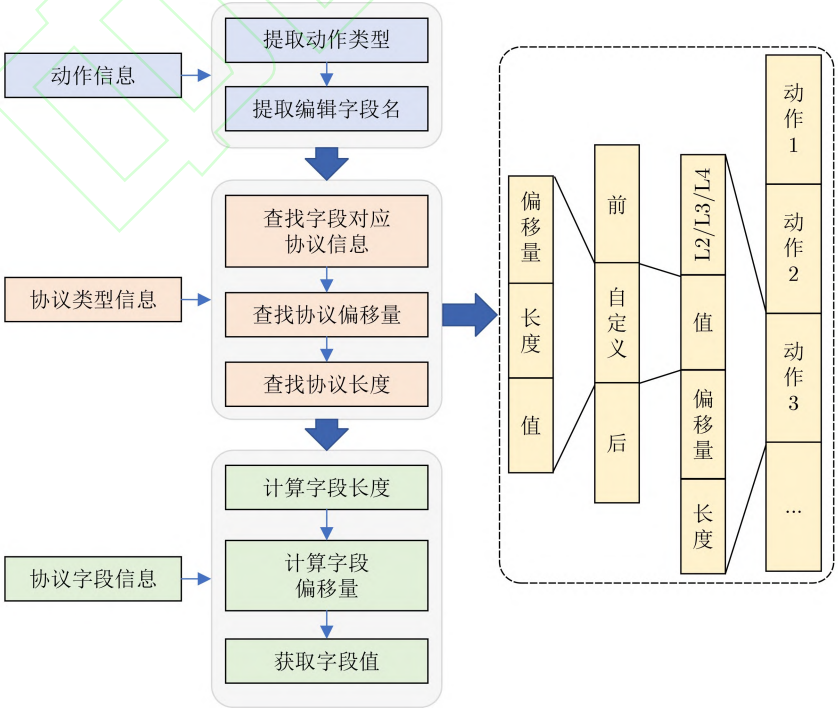


图 5 FlexEdit指令转译过程

表 2　IPv4模态CLI指令

ID	CLI	说明
1	config udf_1 L2 96 0X0800 0XFFFF	etherType通过udf_1这条规则匹配，基础偏移类型L2 偏移96位，掩码0XFFFF，匹配0X0800
2	config udf_2 L2 240 0X0A000003 0XFFFFFFFFF	dst_addr通过udf_2这条规则匹配，基础偏移类型L2 偏移240位，掩码0XFFFFFFFFF，匹配0X0A000003
3	config udf match udf_1 udf_2 action Ethernet2	将匹配到的数据包从Ethernet2端口转发

2.7　p4c-TsingMa的共性技术提炼

本文所提出的p4c-TsingMa可提炼出“解析-映射-优化”3阶编译链路的共性技术框架。无论目标芯片架构(如PISA架构的Intel Tofino、非PISA架构的TsingMa.MX、可编程逻辑架构的FPGA)如何差异，高级网络编程语言(以P4为代表)到芯片可执行命令的编译，均需通过中间表示解析→硬件特性映射→资源约束优化3个核心环节，这是后端编译的本质共性。

中间表示解析方面，本文采用的“先序遍历+节点属性定向提取”方法具有通用性：针对IR的DAG结构，通过Visitor模式遍历协议头(Type_Header)、解析逻辑(ParserState)、匹配表(P4Table)等核心节点，构建协议-字段-元数据的标准化映射。这种按节点类型分类提取、按硬件需求组织信息的方法，可扩展至其他芯片。

硬件特性映射方面，本文针对TsingMa.MX的FlexParser/FlexEdit特性设计的映射逻辑，体现了该环节的共性思路。语义拆解将P4中自定义字段解析语义拆解为字段偏移计算和匹配条件定义两个子任务；参数映射将拆解后的子任务，映射为芯片硬件参数，如FlexParser的基础偏移类型(L2/L3)字段长度，FlexEdit的编辑操作类型(插入/删除)；指令生成按硬件接口格式，如TsingMa.MX的CLI命令、Tofino的bf_switchd配置文件，将参数组合为可执行命令。

资源约束优化方面，本文提出的UDF表项合并算法以资源利用率最大化为目标，通过降序排序(优先处理长规则)、分组填充(短规则填充剩余资源)和字段复用(如多种网络模态共享etherType字段)方法，将多模态规则合并为硬件可承载的指令。这种按资源限制分组、按规则特征复用的策略，也可迁移至其他场景。

3　测试与评估

本文使用C++和Python语言在p4c项目原型上实现了面向盛科TsingMa.MX交换芯片的多模态网络后端编译器p4c-TsingMa。测试环境为搭载盛科TsingMa.MX交换芯片的核心级多模态网元PINE-B1000-P48C8A，该网元运行SONIC操作系统，通过容器的方式来管理和配置芯片的不同类型资源。本文将p4c-TsingMa编译器运行在一个Ubuntu18.04操作系统的容器中，并将该容器部署在的核心级多模态网元上。在实验中，本文将多模态网络数据报文处理作为基准程序，而不是现有P4编译中的网络监控[20]、流量管理[21]、拥塞控制[22]和网络安全等基准程序，这些基准程序并不适用于多模态网络后端编译器性能的评估。考虑到盛科TsingMa.MX芯片与BMv2,FPGA,Tofino等目标平台差异显著，对比缺乏有效性，故本文未将p4c-TsingMa与适配这些平台的编译器开展性能对比。同时，各编译方法优化目标与评价指标维度不同，且目前无同类型国产交换芯片的编译方法可作参照，强行对比需改造其他方法，会偏离其原设计初衷。

3.1　编译性能分析

为了验证后端编译器对不同网络模态转发程序的编译和UDF表项合并编译，本文第1组实验下发不同模态转发的P4程序到核心级网元，通过p4c-TsingMa编译器编译配置TsingMa.MX芯片。本文分析了不同模态处理程序编译之后芯片寄存器资源的使用情况，并与经过UDF表项合并编译之后的寄存器资源使用情况进行比较，来说明UDF表项合并算法的效果。

实验结果如表3所示，前5个模态分别是接入级移动标识模态(Access Mobility First, AccMF)、核心级移动标识模态(Core Mobility First, CoreMF)、IPv4模态、IPv6模态和工控标识模态(PowerLink, PwL)。将单个模态的转发程序在服务器上经过编译

表 3　芯片资源占用情况

网络模态	关键字个数	关键字长度(bit)	寄存器资源利用率(%)
AccMF	3	80	62.5
CoreMF	2	32	25
IPv4	2	48	37.5
PwL	2	64	50
IPv6	2	144	100
AccMF_CoreMF_IPv4_PwL	6	176	100

之后，下发到多模态网元，经过实际的编译测试之后，不同网络模态的转发程序均能正确下发到TsingMa.MX交换芯片。并且，根据芯片寄存器的占用率可以看出，除了IPv6网络模态，其余模态程序的编译之后的寄存器占用率均没有达到100%。从实验结果发现，经过UDF表项合并后，可提升寄存器资源利用率37.5%～75%，使TsingMa.MX交换芯片同时处理多种网络模态报文的解析与转发。

3.2 编译结果验证

为了验证经过编译部署之后的芯片能否正常处理多模态数据报文，本文搭建了一个简易的网络测试环境。如图6所示，测试环境包括1台多模态网元设备和2台高性能服务器。这两台服务器均配备了多个以太网端口，能够模拟多模态网络环境中的多种设备。其中一台服务器作为发包终端，负责生成和发送多模态数据包；另一台服务器作为接收终端，负责接收和处理这些数据包。本实验验证不同模态的数据报文，经过多模态网元之后，能否被正确转发到流表文件中对应的接收端口。

本文使用多模态网络数据报文转发的基准程序，将网络模态程序输入到编译器，经过编译配置到芯片中。结合控制器下发的流表文件如图7所示。发包终端通过其以太网端口发送不同模态数据报文到多模态网元的1号端口，接收终端通过其以太网5端口和以太网6端口接收不同模态的数据报文。为了验证这一过程，本文使用Wireshark工具分别在发送终端和接收终端进行监控。在发送终端，使用Wireshark监控以太网端口，捕获发送的数据包；在接收终端，使用Wireshark监控以太网5端口和以太网6端口，捕获接收的数据包。通过Wireshark抓包工具对数据包进行捕获与分析，实验结果如图8所示。在100 s之前，还未进行编译配置，芯片会将没有UDF命中的数据报文广播至所有端口，所以两个端口都捕获到了五种模态的数据报文。从100 s开始，芯片编译配置成功，经过观察发现，芯片能够根据UDF的内容准确识别不同模态的数据报文，并能够按照预期将不同模态数据报文的线速转发至流表指定的端口。这一实验结果验证了编译系统生成的配置信息可有效指导芯片执行数据报文转发操作。

4 结束语

本文设计了面向国产交换芯片的多模态网络后端编译器p4c-TsingMa。本编译器通过先序遍历解析DAG结构的中间表示，精准提取协议类型等关键信息；结合芯片FlexParser与FlexEdit特性完成指令转译，实现P4程序到芯片控制命令的映射；提出UDF表项合并算法，解决寄存器资源有限问题。实验验证，p4c-TsingMa可正确编译多模态程序并实现报文精准转发，相较于未启用UDF表项合并算法的场景，寄存器资源利用率提升37.5%～75%。

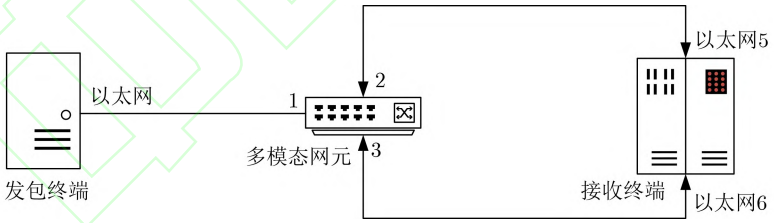
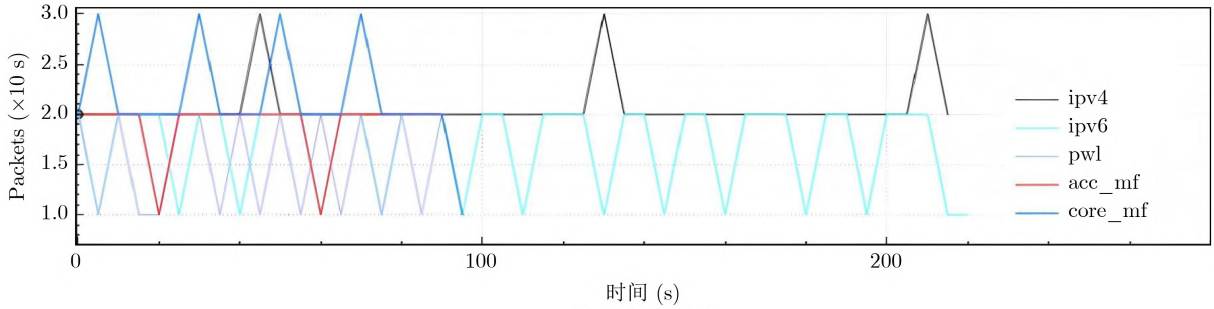


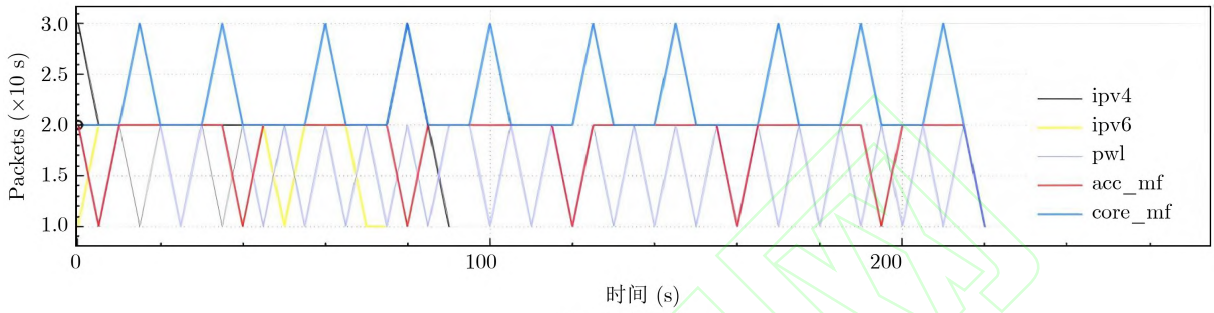
图 6 测试环境拓扑

```
### s1
p4 = bfrt.ipv4_ipv6_pwl_8180_asic.pipe
#将目的IPv4地址是10.0.0.4数据包从端口3转发出去
p4.IngressPipeImpl.ipv4_exact.add_with_ipv4_set_egress_port(dst_addr=0x0A000004, port=3)
#0x1234000000000000005678901234567890是1234::5678:9012:3456:7890的16进制表示
#将目的IPv6地址是1234::5678:9012:3456:7892数据包从端口3转发出去
p4.IngressPipeImpl.ipv6_exact.add_with_ipv6_set_egress_port(dst_addr=0x1234000000000000005678901234567892, port=3)
#将目的mac地址是12:34:56:78:90:11数据包从端口2转发出去
p4.IngressPipeImpl.pwl_exact.add_with_pwl_set_egress_port(dstAddr=0x123456789011, port=2)
#接入网，此表在ASIC判断是0x9000后执行
#将src_aid是101(0x65),dst_aid是201(0xc9)的数据包从端口2转发出去
p4.IngressPipeImpl.accmf_exact.add_with_accmf_set_egress_port(src_aid=101,dst_aid=201, port=2)
#核心网，此表在ASIC判断是0x9300后执行
#将dst_rid是1002(0x03ea)的数据包从端口2转发出去
p4.IngressPipeImpl.coremf_exact.add_with_coremf_set_egress_port(dst_rid=1002, port=2)
```

图 7 流表文件



(a) 以太网端口5



(b) 以太网端口6

图 8 端口流量分析图

参 考 文 献

[1] 涂化清, 廖君虎, 朱俊, 等. 多模态网络环境下网络模态共存与优化部署方法[J]. 电子学报, 2025, 53(5): 1650–1660. doi: 10.12263/DZXB.20250015.
TU Huaqing, LIAO Junhu, ZHU Jun, *et al.* Network modal coexistence and optimal deployment method in polymorphic network environment[J]. *Acta Electronica Sinica*, 2025, 53(5): 1650–1660. doi: 10.12263/DZXB.20250015.

[2] WU Jiangxing, LI Junfei, SUN Penghao, *et al.* Theoretical framework for a polymorphic network environment[J]. *Engineering*, 2024, 39: 222–234. doi: 10.1016/j.eng.2024.01.018.

[3] 邬江兴, 胡宇翔. 网络技术体系与支撑环境分离的发展范式[J]. 信息通信技术与政策, 2021, 47(8): 1–11. doi: 10.12267/j.issn.2096-5931.2021.08.001.
WU Jiangxing and HU Yuxiang. The development paradigm of separation between network technical system and supporting environment[J]. *Information and Communications Technology and Policy*, 2021, 47(8): 1–11. doi: 10.12267/j.issn.2096-5931.2021.08.001.

[4] HU Yuxiang, LI Dan, SUN Penghao, *et al.* Polymorphic smart network: An open, flexible and universal architecture for future heterogeneous networks[J]. *IEEE Transactions on Network Science and Engineering*, 2020, 7(4): 2515–2525. doi: 10.1109/tnse.2020.3006249.

[5] BOSSHART P, DALY D, GIBB G, *et al.* P4: Programming protocol-independent packet processors[J]. *ACM SIGCOMM Computer Communication Review*, 2014, 44(3): 87–95. doi: 10.1145/2656877.2656890.

[6] Tofino[EB/OL]. https://www.barefootnetworks.com/products/brief-tofino, 2025.

[7] Centec. CTC8180[EB/OL]. https://www.centec.com/silicon/26, 2025.

[8] HAUSER F, HÄBERLE M, MERLING D, *et al.* A survey on data plane programming with P4: Fundamentals, advances, and applied research[J]. *Journal of Network and Computer Applications*, 2023, 212: 103561. doi: 10.1016/j.jnca.2022.103561.

[9] YANG Y Fifan, HE Lin, ZHOU Jiasheng, *et al.* P4runpro: Enabling runtime programmability for RMT programmable switches[C]. The ACM SIGCOMM 2024 Conference, Sydney, Australia, 2024: 921–937. doi: 10.1145/3651890.3672230.

[10] CHEN Zhikang, FENG Yong, LIU Shuxin, *et al.* OptimusPrime: Unleash dataplane programmability through a transformable architecture[C]. The ACM SIGCOMM 2024 Conference, Sydney, Australia, 2024: 904–920. doi: 10.1145/3651890.3672214.

[11] WANG Tao, YANG Xiangrui, ANTICHI G, *et al.* Isolation mechanisms for high-speed packet-processing pipelines[C]. The 19th USENIX Symposium on Networked Systems Design and Implementation, Renton, USA, 2022: 1289–1305.

[12] LU Chenyunfei, TANG Zhu, PENG Wei, *et al.* Running P4 programs on general programmable network interconnection chips[C]. 2023 Fourth International Conference on Frontiers of Computers and Communication Engineering, Xiamen, China, 2023: 1–6. doi: 10.1109/fcce58525.2023.00008.

[13] DA ROBIN D and KHAN J I. An open-source $P4_{16}$

compiler backend for reconfigurable match-action table switches: Making networking innovation accessible[J]. *Computer Networks*, 2024, 242: 110246. doi: 10.1016/j.comnet.2024.110246.

[14] GAO Jiaqi, ZHAI Ennan, LIU H H, *et al*. Lyra: A cross-platform language and compiler for data plane programming on heterogeneous ASICs[C]. The Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, 2020: 435–450. doi: 10.1145/3387514.3405879.

[15] LI Yifan, GAO Jiaqi, ZHAI Ennan, *et al*. Cetus: Releasing P4 programmers from the chore of trial and error compiling[C].The 19th USENIX Symposium on Networked Systems Design and Implementation, Renton, USA, 2022: 371–385.

[16] DANGETI T K, KEERTHY S V, and UPADRASTA R. P4LLVM: An LLVM based P4 compiler[C]. The 26th International Conference on Network Protocols, Cambridge, UK, 2018: 424–429. doi: 10.1109/icnp.2018.00059.

[17] WANG Han, SOULÉ R, DANG H T, *et al*. P4FPGA: A rapid prototyping framework for P4[C]. The Symposium on SDN Research, Santa Clara, USA, 2017: 122–135. doi: 10.1145/3050220.3050234.

[18] GAO Xiangyu, KIM T, WONG M D, *et al*. Switch code generation using program synthesis[C]. The Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication, 2020: 44–61. doi: 10.1145/3387514.3405852.

[19] p4c, a reference compiler for P4 programming language[EB/OL]. https://github.com/p4lang/p4c, 2025.

[20] VENKATAKEERTHY S, AGGARWAL R, JAIN S, *et al*. IR2VEC: LLVM IR based scalable program embeddings[J]. *ACM Transactions on Architecture and Code Optimization (TACO)*, 2020, 17(4): 32. doi: 10.1145/3418463.

[21] HARKOUS H, PAPAGIANNI C, DE SCHEPPER K, *et al*. Virtual queues for P4: A poor man's programmable traffic manager[J]. *IEEE Transactions on Network and Service Management*, 2021, 18(3): 2860–2872. doi: 10.1109/tnsm.2021.3077051.

[22] VENKATAKEERTHY S, ANDALURI Y, DEY S, *et al*. Packet processing algorithm identification using program embeddings[C]. The 6th Asia-Pacific Workshop on Networking, Fuzhou, China, 2022: 76–82. doi: 10.1145/3542637.3542649.

涂化清：女，博士后，研究方向为软件定义网络、可编程数据平面、多模态网络等.

王元鸿：男，硕士，研究方向为多模态网络、可编程网络.

徐　琪：男，副研究员，研究方向为多模态网络、软件定义网络、时间敏感网络.

朱　俊：男，高级工程师，研究方向为新型网络体系结构、软件定义网络、网络资源管理、网络协议设计优化等.

邹　涛：男，研究员，研究方向为多模态网络技术和新型网络体系结构.

隆克平：男，教授，研究方向为新一代网络技术、光互联网关键技术、无线通信技术、人工智能与大数据.

责任编辑：余　蓉

A Polymorphic Network Backend Compiler for Domestic Switching Chips

TU Huaqing① WANG Yuanhong② XU Qi① ZHU Jun①
ZOU Tao① LONG Keping①

①(*Zhejiang Laboratory*, *Hangzhou* 310000, *China*)

②(*Institute of Computing Technology Chinese Academy of Sciences*, *Beijing* 100000, *China*)

Abstract:

Objective The P4 language and programmable switching chips offer a feasible approach for deploying polymorphic networks. However, polymorphic network packets written in P4 cannot be directly executed on the domestically produced TsingMa.MX programmable switching chip developed by Centec, which necessitates the design of a specialized compiler to translate and deploy the P4 language on this chip. Existing backend compilers are mainly designed and optimized for software-programmable switches such as BMv2, FPGAs, and Intel Tofino series chips, rendering them unsuitable for compiling polymorphic network programs for the TsingMa.MX chip. To resolve this limitation, a backend compiler named p4c-TsingMa is proposed for the TsingMa.MX switching chip. This compiler enables the translation of high-level network programming

languages into executable formats for the TsingMa.MX chip, thereby supporting the concurrent parsing and forwarding of multiple network modal packets.

Methods p4c-TsingMa first employs a preorder traversal approach to extract key information, including protocol types, protocol fields, and actions, from the Intermediate Representation (IR). It then performs instruction translation to generate corresponding control commands for the TsingMa.MX chip. Additionally, p4c-TsingMa adopts a User Defined Field (UDF) entry merging method to consolidate matching instructions from different network modalities into a unified lookup table. This design enables the extraction of multiple modal matching entries in a single operation, thereby enhancing chip resource utilization.

Results and Discussions The p4c-TsingMa compiler is implemented in C++, mapping network modal programs written in the P4 language into configuration instructions for the TsingMa.MX switching chip. A polymorphic network packet testing environment (Fig. 7) is established, where multiple types of network data packets are simultaneously transmitted to the same switch port. According to the configured flow tables, the chip successfully identifies polymorphic network data packets and forwards them to their corresponding ports (Fig. 9). Additionally, the table entry merging algorithm improves register resource utilization by 37.5% to 75%, enabling the chip to process more than two types of modal data packets concurrently.

Conclusions A polymorphic network backend compiler, p4c-TsingMa, is designed specifically for domestic switching chips. By utilizing the FlexParser and FlexEdit functions of the TsingMa chip, the compiler translates polymorphic network programs into executable commands for the TsingMa.MX chip, enabling the chip to parse and modify polymorphic data packets. Experimental results demonstrate that p4c-TsingMa achieves high compilation efficiency and improves register resource utilization by 37.5% to 75%.

Key words: Polymorphic network; Programmable data plane; Domestic switching chips; Backend compiler